

Nombre y apellido:LU:

E-mail:# Hojas:Firma:

1a	1b	2	3a	3b	3c	4a	4b	4c	Nota

Incluir nombre, fecha y firma en todas las hojas.

Ejercicio 1 (Razonamiento ecuacional). Considerar las siguientes definiciones en Haskell:

{N0} nand1 b [] = b

{N1} nand1 b (x:xs) = nand1 (not x || b) xs

{A0} any f [] = False

{A1} any f (x:xs) = f x || any f xs

Demostrar la siguiente propiedad: nand1 False = any not

Se pueden utilizar las propiedades conocidas de || y not mencionándolas al momento de usarlas.

Ejercicio 2 (Deducción natural). Extendemos el sistema de deducción natural con un nuevo conectivo lógico binario Δ , que tiene las siguientes reglas de introducción y eliminación:

$$\frac{\Gamma, \tau, \sigma \vdash \perp}{\Gamma \vdash \tau \Delta \sigma} \Delta_i \quad \frac{\Gamma \vdash \tau \Delta \sigma \quad \Gamma, \neg \tau \vdash \rho \quad \Gamma, \neg \sigma \vdash \rho}{\Gamma \vdash \rho} \Delta_e$$

a) Demostrar que vale $\vdash \neg(\tau \wedge \sigma) \Rightarrow \tau \Delta \sigma$ para todo par de fórmulas τ y σ .b) Demostrar que vale $\tau \Delta \sigma \vdash \neg(\tau \wedge \sigma)$ para todo par de fórmulas τ y σ .**Ejercicio 3 (Cálculo Lambda).** Dos programas M y N de un mismo tipo son *distinguishables* si existen valores $V_0, V_1, \dots, V_n$ con $n \geq 0$ tales que $V_0(MV_1 \dots V_n) \rightarrow \text{True}$ y $V_0(NV_1 \dots V_n) \rightarrow \text{False}$ (o viceversa, lo cual es equivalente), e *indistinguishables* en caso contrario.

Indicar si cada una de las siguientes afirmaciones es verdadera o falsa y justificar:

a) Si existe M' tal que $M \rightarrow M'$ y $N \rightarrow M'$, entonces M y N son indistinguishables.b) Si existen V y V' tales que $MV \rightarrow V'$ y $NV \rightarrow V'$, entonces M y N son indistinguishables.c) Si $\vdash M : \tau \rightarrow \sigma$ es derivable, entonces M y $\lambda x : \tau. Mx$ son indistinguishables.**Ejercicio 4 (Programación lógica).** Se cuenta con el siguiente predicado para verificar que una fórmula en Lógica Proposicional esté bien formada:

```
%formula(+F)
formula(X) :- atom(X).
formula(not(P)) :- formula(P).
```

```
formula(and(P,Q)) :- formula(P), formula(Q).
formula(or(P,Q)) :- formula(P), formula(Q).
```

a) Definir el predicado `formula(+XS, -F)` que, dada una lista de átomos XS sin elementos repetidos, vaya instanciando en F todas las fórmulas cuyas variables proposicionales pertenecen a XS .

b) El predicado anterior, ¿es reversible en su segundo parámetro? Justificar explicando qué sucede si se pasa la fórmula instanciada.

c) Se dispone de un predicado `valuation(+F, -V)` que va instanciando en V todas las valuaciones que asignan valores de verdad a las variables de la fórmula F , y un predicado `eval(+V, +F)` que nos da el resultado de evaluar a F en la valuación V (la representación de las valuaciones no es importante).Definir el predicado `tautology(+F)`, que tenga éxito si F es una tautología (tener en cuenta que para que algo sea una tautología debe ser una fórmula).